

Sun'iy intellektda muammolarni hal qilish

Muammolarni hal qilish sun'iy intellektning (AI) asosiy tushunchasi bo'lib, unda tizimlar muammolarni aniqlash, qarorlar qabul qilish va samarali echimlarni topish uchun mo'ljallangan. AI agentlardan foydalanadi, ular atrof-muhitni idrok etadigan va aniq maqsadlarga erishish uchun harakatlarni amalga oshiradi. Ular ogohlantirishlarga oldindan belgilangan qoidalar asosida reaksiyaga kirishadigan oddiy refleks vositalaridan tashqariga chiqadilar. Buning o'rniga, muammoni hal qiluvchi agentlar vaziyatlarni faol ravishda tahlil qiladilar, turli variantlarni baholaydilar va maqsadlariga erishish uchun eng yaxshi harakatni tanlaydilar.

Ushbu agentlar tomonidan ishlaydi:

- **Atrof-muhitni idrok etish:** Ular sensorli kirishlar yoki kuzatuvlar kabi atroflari haqida ma'lumot to'playdi.
- **Muammoni aniqlash:** ular muammoni, shu jumladan boshlang'ich nuqtasini, mavjud harakatlarni va kerakli maqsadni aniq tushunadilar.
- **Turli xil imkoniyatlarni o'rganish:** Ular muammoni hal qilishning turli usullarini ko'rib chiqadilar va qaysi yondashuv muvaffaqiyat qozonishini baholaydilar.
- **Baholash va qaror qabul qilish:** Variantlarni o'rgangach, ular natijalarni baholaydilar va vaqt, resurslar va muvaffaqiyat ehtimoli kabi omillarga asoslanib, eng yaxshi harakat yo'nalishini tanlaydilar.
- **O'rganish va moslashish:** Ko'pgina muammolarni hal qilish agentlari vaqt o'tishi bilan qaror qabul qilish qobiliyatlarini yaxshilagan holda o'tmishdagi tajribalardan o'rganishlari mumkin.

Aldagi muammolar turlari

AI tizimlari har xil turdagi muammolarga duch kelishi mumkin. Ushbu muammolar tizimning ishlashiga ta'siri va ularni qanday hal qilish mumkinligi asosida tasniflanadi:

1. E'tiborsiz muammolar

Bular AI tizimining umumiy ishlashiga kam yoki umuman ta'sir qilmaydigan kichik muammolar. Ular ko'pincha zararsizdir va darhol e'tibor talab qilmaydi.

Misollar :

- Yakuniy natijaga ta'sir qilmaydigan prognozlardagi kichik noaniqliklar, masalan, tasvirni tasniflashdagi engil xato.
- Natijani o'zgartirmaydigan ma'lumotlarni oldindan qayta ishlashdagi kichik muammolar.

Ishlov berish : e'tiborga olinmaydi, chunki ular tizimning asosiy funktsiyasiga xalaqit bermaydi.

2. Qayta tiklanadigan muammolar

Qayta tiklanadigan muammolar, AI tizimi muammoga duch kelgan, ammo xatolarni boshqarish funktsiyalari kabi avtomatik yoki qo'lda aralashuv bilan hal qilinishi mumkin bo'lgan muammolar.

Misollar :

- Statistik usullar yordamida to'ldirilishi mumkin bo'lgan etishmayotgan ma'lumotlar.
- Zaxiradan tiklash orqali tuzatilishi mumkin bo'lgan tizim ishdan chiqishi.

Ishlov berish : xatolarni avtomatlashtirilgan hal qilish, modelni qayta o'qitish yoki nazorat punktlari orqali tizimni tiklash kabi harakatlarni talab qiladi.

3. Qayta tiklab bo'lmaydigan muammolar

Bu jiddiy muammolar bo'lib, tizimni qayta tiklashni imkonsiz qiladigan doimiy shikastlanish yoki nosozliklar. Ular sezilarli ish faoliyatini yo'qotishiga olib kelishi mumkin.

Misollar :

- Buzuq ta'lim ma'lumotlari noto'g'rilikka olib keladi va model samaradorligini pasaytiradi.
- Modelni ishonchsiz qiladigan raqib hujumlari.
- Model haddan tashqari ixtisoslashgan bo'lib, yangi ma'lumotlarga moslasha olmaydi.

Ishlov berish : tizimni to'liq qayta qurish, shu jumladan modelni qayta o'qitish yoki asosiy muammolarni bartaraf etish uchun ma'lumotlarni yangilashni talab qiladi.

Alda muammolarni hal qilish bosqichlari

AI muammosini hal qilish inson tafakkuriga o'xshash tuzilgan, mantiqiy jarayonni kuzatib boradi, uning bosqichlari quyidagilardan iborat:

1. **Muammoni aniqlash:** Birinchidan, muammoni aniq belgilash kerak. Bunga kirishlar, cheklovlar va yechim qanday bo'lishi kerakligini tushunish kiradi.
2. **Muammoni tahlil qilish:** Muammo aniqlangach, u batafsilroq tahlil qilinadi. Bu uning cheklovlari va mumkin bo'lgan echimlarini tushunishga yordam beradi.
3. **Bilimlarni taqdim etish:** Barcha muhim ma'lumotlar sun'iy intellektni tushunishi va u bilan ishlashi uchun tashkil etilgan. Bu grafiklarni yaratish yoki ma'lumotlar bazalaridan foydalanishni o'z ichiga olishi mumkin.
4. **Muammoni hal qilish:** AI muammoni hal qilish uchun tegishli usullardan foydalanadi. Bu ko'pincha eng samarali strategiyani topish uchun turli strategiyalarni solishtirishni anglatadi.
5. **Sinov va baholash:** Nihoyat, yechim amalga oshirilgandan so'ng, sinov va baholash yechimning barcha talablarga javob berishini va kutilganidek ishlashini ta'minlaydi.

AIda muammolarni shakllantirishning asosiy komponentlari

AIda muammolarni samarali hal qilish bir nechta muhim komponentlarga bog'liq:

1. **Dastlabki holat:** Bu AI o'z jarayonini boshlagan muammoning boshlang'ich nuqtasidir. U kontekstni belgilaydi va agent muammoga qanday yondashishini aniqlashga yordam beradi.
2. **Harakat:** Ushbu bosqichda AI boshlang'ich holatdan amalga oshirishi mumkin bo'lgan barcha mumkin bo'lgan harakatlarni aniqlaydi. Har bir harakat tizimning muammoni hal qilishga qanchalik yaqinlashishiga ta'sir qiladi.
3. **O'tish:** Bu tizim qanday harakat qilinganidan keyin bir holatdan ikkinchi holatga o'zgarishini anglatadi. O'tish davrini modellash agentning harakatlari keyingi bosqichlarga qanday ta'sir qilishini ko'rsatishga yordam beradi.
4. **Maqsad testi:** Harakat qilingandan so'ng, AI uning maqsadiga erishganligini tekshiradi. Maqsadga erishilsa, muammoni hal qilish jarayoni to'xtaydi va yechim tugallangan hisoblanadi.
5. **Xarajat funktsiyasi:** Bu qadam maqsadga erishish uchun sarflanadigan xarajatlarga raqamli qiymat beradi. Xarajat vaqt, energiya yoki pul kabi resurslarni o'z ichiga olishi mumkin va maqsadga erishishning eng samarali usulini tanlashga yordam beradi.

AIda muammolarni hal qilish texnikasi

AIda turli xil muammolarni hal qilish uchun bir nechta texnikalar mavjud. Texnikani tanlash muammoning tabiatiga va mavjud resurslarga bog'liq.

1. Qidiruv algoritmlari

Qidiruv algoritmlari mumkin bo'lgan echimlarni o'rganish uchun muhimdir. Ikkita asosiy toifa mavjud:

- **Axborotsiz qidiruv** : Bu algoritmlar, masalan, kenglik-birinchi va chuqur-birinchi qidiruv, muammo haqida qo'shimcha ma'lumotdan foydalanmaydi va barcha mumkin bo'lgan yo'llarni o'rganmaydi.
- **Axborotlangan qidiruv** : A* kabi algoritmlar qidiruvni yanada istiqbolli yo'llarga yo'naltirish uchun evristika yoki domenga oid bilimlardan foydalanadi.

2. Cheklovlarni qondirish muammolari (CSP)

CSPlarda maqsad bir qator cheklovlarni qondiradigan echimlarni topishdir. Yechim maydonini samarali o'rganish va barcha cheklovlar bajarilishini ta'minlash uchun orqaga qaytish, cheklovlarni ko'paytirish va mahalliy qidiruv kabi usullar qo'llaniladi.

3. Optimallashtirish texnikasi

AI ko'pincha optimallashtirish muammolarini hal qiladi, bu erda maqsad ko'plab variantlar orasida eng yaxshi echimni topishdir. Texnikalarga quyidagilar kiradi:

1. Chiziqli munosabatlarga oid masalalarni yechish uchun chiziqli dasturlash.
2. Murakkab muammolarni kichikroq kichik muammolarga bo'lish uchun dinamik dasturlash.
3. Optimal echimlarni topish uchun tabiiy tanlanishni taqlid qiluvchi genetik algoritmlar kabi evolyutsion algoritmlar.

4. Mashinada o'rganish

Mashinani o'rganish AI tizimlariga ma'lumotlardan o'rganish va vaqt o'tishi bilan yaxshilash imkonini beradi. Ta'limning uchta asosiy turi:

1. Nazorat ostida o'rganish: bashorat qilish yoki tasniflash uchun etiketli ma'lumotlardan o'rganish.
2. Nazoratsiz o'rganish: Belgilangan natijalarsiz ma'lumotlardagi naqshlarni aniqlash.
3. O'rganishni mustahkamlash: Sinov va xato orqali o'rganish, mukofot yoki jazo shaklida fikr-mulohazalarni qabul qilish.

5. Tabiiy tilni qayta ishlash (NLP)

Tabiiy tilni qayta ishlash (NLP) AI tizimlariga inson tilini tushunish va yaratishga yordam beradi. Chatbotlar, til tarjimai va matnni umumlashtirish uchun tokenizatsiya, nutqning bir qismi (PoS) va nomli ob'ektni tanib olish kabi usullar muhim ahamiyatga ega.

Muammolarni hal qilishda AIning afzalliklari

1. **Samaradorlik** : U muammolarni odamlarga qaraganda tezroq hal qila oladi, katta hajmdagi ma'lumotlarni qayta ishlaydi va naqshlarni tezda aniqlaydi.
2. **Aniqlik** : To'g'ri ma'lumotlar va modellar bilan AI yuqori aniqlik bilan qaror qabul qilishi mumkin, ayniqsa tibbiy tashxis yoki tasvirni aniqlash kabi vazifalarda.
3. **Avtomatlashtirish** : U takrorlanuvchi vazifalarni avtomatlashtiradi, murakkabroq va ijodiy ishlar uchun inson vaqtini bo'shatadi.
4. **Moslashuvchanlik** : AI tizimlari, xususan, mashinani o'rganishdan foydalanadigan tizimlar vaqt o'tishi bilan yaxshilanishi, yangi ma'lumotlarga moslashishi va o'zgaruvchan muammolar.
5. **Iqtisodiy samaradorlik** : Amalga oshirilgandan so'ng, u operatsiyalarni soddalashtirish, samaradorlikni oshirish va inson xatolarini minimallashtirish orqali xarajatlarni kamaytirishi mumkin.

AI muammolarini hal qilishdagi qiyinchiliklar

Ko'pgina afzalliklarga qaramay, u bir qator qiyinchiliklarga duch keladi:

1. **Murakkablik** : Ba'zi muammolar juda murakkab bo'lib, hal qilish uchun katta hisoblash resurslari va vaqt talab etiladi.
 2. **Ma'lumotlar sifati** : AI tizimlari ma'lumotlar sifatiga bog'liq va yomon yoki noto'g'ri ma'lumotlar noto'g'ri yoki noto'g'ri natijalarga olib kelishi mumkin.
 3. **Interpretability** : Ko'pgina AI modellari, ayniqsa chuqur o'rganish, shaffoflikdan mahrum bo'lib, qarorlar qanday qabul qilinishini tushunishni qiyinlashtiradi.
 4. **Etika va tarfkashlik** : AI adolatsiz yoki axloqiy bo'lmagan natijalarga olib keladigan ma'lumotlarning noto'g'riligini saqlab qolishi mumkin, adolatni ta'minlash asosiy muammodir.
- Sun'iy intellekt rivojlanishda davom etar ekan, muammolarni hal qilish qobiliyatini oshirish texnologiyani rivojlantirish va inson tajribasini yaxshilash uchun muhim bo'lib qoladi.

Yangi boshlanuvchilar uchun qidiruv algoritmlari yordamida muammoni yechishga kirish

Kirish

Kompyuter fanida muammoni hal qilish sun'iy intellekt usullariga, jumladan, yashil algoritmlarni shakllantirish, evristika va echimlarni topish uchun asosiy sabablarni tahlil qilishni anglatadi. Qidiruv algoritmlari informatika sohasidagi ko'plab muammolarni hal

qilish uchun asosiy vositadir. Ular muqobil variantlarni qiyin va tezkor o'rganish orqali javoblarni topishning tizimli usulini taqdim etadi. Ushbu algoritmlar turli xil ilovalarda yo'lni aniqlash, rejalashtirish va ma'lumotlarni qidirishda qo'llaniladi. Ushbu maqolada biz turli xil qidiruv algoritmlari tomonidan qo'llaniladigan muammolarni hal qilish usullarini ko'rib chiqamiz.

Umumiy ko'rinish:

- AIDA izlash orqali muammolarni hal qilishning roli va uning murakkab hisoblash muammolarini hal qilishdagi ahamiyati haqida bilib oling.
- Qidiruv algoritmlarining xossalari va ular qidiruv jarayonini optimal echimlar sari qanday boshqarishi haqida tushunchaga ega bo'ling.
- Qidiruv algoritmlari tomonidan qo'llaniladigan muammolarni hal qilishning turli usullarini, shu jumladan ma'lumotsiz va ma'lumotli yondashuvlarni o'rganing.
- Qidiruv algoritmlarining turlarini o'rganing, ulardan foydalanish holatlari va yo'llarni aniqlash, rejalashtirish va ma'lumotlarni qidirish kabi sohalardagi ilovalarga e'tibor qarating.
- Turli ma'lumotga ega bo'lmagan qidiruv algoritmlarini va ularning samaradorligiga asoslangan holda ma'lumotli qidiruv algoritmlari bilan qanday farq qilishini solishtiring.
- Axborotsiz va ma'lumotli qidiruv algoritmlarining afzalliklari va cheklovlarini ko'rib chiqing, ularning yondashuv va samaradorlikdagi farqlarini ko'rsating.
- AI muammolarini hal qilishning aniq stsenariylari uchun mos algoritmni tanlashning asosiy yo'nalishlarini tushuning.

Sun'iy intellektidagi muammolarga misollar

Bugungi tez sur'atda raqamlashtirilgan dunyoda sun'iy intellekt usullari resurslar va vaqtdan samarali foydalana oladigan tizimlarni avtomatlashtirish uchun keng qo'llaniladi. Kundalik hayotda boshdan kechirilgan taniqli muammolardan ba'zilari o'yinlar va boshqotirmalardir. AI texnikasidan foydalangan holda biz ushbu muammolarni samarali hal qilishimiz mumkin. Shu ma'noda, AI tomonidan hal qilinadigan eng keng tarqalgan muammolardan ba'zilari:

- Sayohatchi sotuvchi muammosi
- Xanoy minorasi muammosi
- Suv idishi muammosi
- N-Queen muammosi
- Shaxmat
- Sudoku
- Kript-arifmetik masalalar
- Sehrli kvadratlar
- Mantiqiy jumboqlar va boshqalar.

Muammolarni yechish usullari

Sun'iy intellektda muammolarni qidirish algoritmlari, evolyutsion hisoblar, bilimlarni namoyish qilish va boshqalar yordamida hal qilish mumkin. Men muammolarni hal qilish uchun ishlatiladigan turli xil qidiruv usullarini muhokama qilaman. Umuman olganda, qidiruv insonga kerakli ma'lumotni topish deb ataladi.

Qidiruv yordamida muammoni yechish jarayoni quyidagi bosqichlardan iborat.

- Muammoni aniqlang
- Muammoni tahlil qiling

- Mumkin bo'lgan echimlarni aniqlash
- Optimal yechimni tanlash
- Amalga oshirish

Keling, qidiruv algoritmlarining ba'zi muhim xususiyatlarini muhokama qilaylik.

Qidiruv algoritmlarining xossalari

Keling, AIDA qidiruv algoritmlarining xususiyatlarini ko'rib chiqaylik.

To'liqlik

Qidiruv algoritmi yechimni berganda yoki berilgan tasodifiy kiritish uchun istalgan yechimni qaytarsa, to'liq hisoblanadi.

Optimallik

Agar topilgan yechim barcha aniqlangan yechimlar ichida eng yaxshisi (eng past yo'l narxi) bo'lsa, u holda bu yechim optimal deb ataladi.

Vaqtning murakkabligi

Algoritmning topshiriqni bajarish uchun sarflagan vaqti vaqt murakkabligi deyiladi. Agar algoritm qisqa vaqt ichida vazifani bajarsa, u samarali bo'ladi.

Kosmik murakkablik

Bu algoritm qidiruv paytida oladigan maksimal xotira yoki xotiradir.

Bu xususiyatlar turli xil qidiruv algoritmlarining samaradorligini solishtirish uchun ham qo'llaniladi.

Qidiruv algoritmlarining turlari

Keling, qidiruv algoritmlarining turlarini ko'rib chiqaylik.

Qidiruv muammolariga asoslanib, biz qidiruv algoritmini quyidagicha tasniflashimiz mumkin

- Axborotsiz qidiruv
- Ma'lumotli qidiruv

Axborotsiz qidiruv algoritmlari

Axborotsiz qidiruv algoritmi yaqinlik, maqsad holatining joylashuvi va h.k. kabi hech qanday domen bilimiga ega emas. U shafqatsiz kuch bilan harakat qiladi. U faqat berilgan daraxtni qanday aylanib o'tishni va maqsad holatini topishni biladi. Ushbu algoritm Blind qidiruv algoritmi yoki qo'pol kuch algoritmi sifatida ham tanilgan. Axborotsiz qidiruv strategiyasining olti turi mavjud.

Ular:

- Kenglik - birinchi qidiruv
- Chuqurlik - birinchi qidiruv
- Chuqurlik cheklangan qidiruv
- Takroriy chuqurlashtirish chuqurligi-birinchi qidiruv
- Ikki tomonlama qidiruv
- Yagona xarajatlarni qidirish

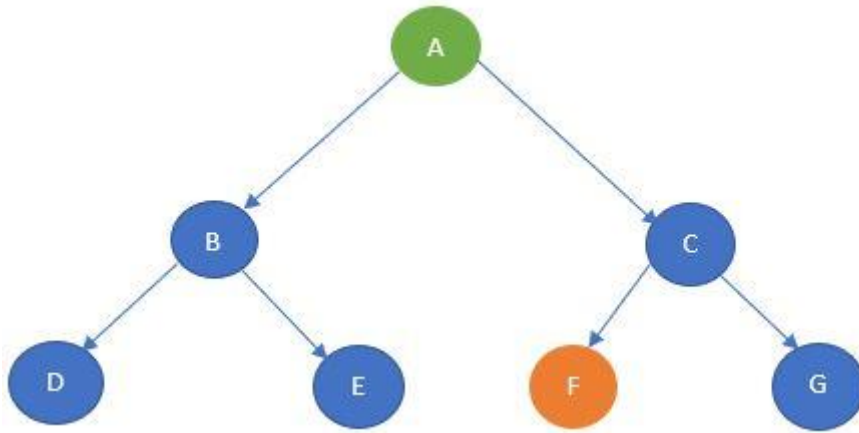
Keling, ushbu oltita strategiyani birma-bir muhokama qilaylik.

1. Kenglik-birinchi qidiruv

Bu eng keng tarqalgan qidiruv strategiyalaridan biridir. Kenglikdagi birinchi qidiruv odatda ildiz tugunidan boshlanadi, qo'shni tugunlarni tekshiradi va keyin keyingi bosqichga o'tadi. U "Birinchi kir, birinchi chiq" (FIFO) strategiyasidan foydalanadi, chunki u yechimga erishish uchun eng qisqa yo'lni beradi.

Berilgan muammo juda kichik bo'lgan va fazoviy murakkablik hisobga olinmagan hollarda BFS qo'llaniladi.

Endi quyidagi daraxtni ko'rib chiqing.



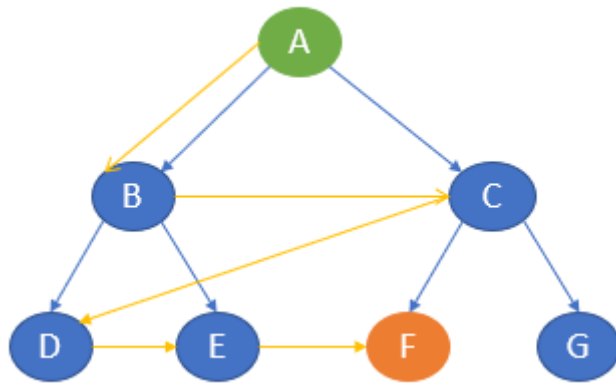
Manba: Muallif

Boshlanish holati sifatida A tugunni, maqsad holati sifatida F tugunni olaylik.

BFS algoritmi boshlang'ich holatidan boshlanadi, keyingi bosqichga o'tadi va maqsad holatiga yetguncha har bir tugunga tashrif buyuradi.

Bu misolda u A dan boshlanadi va keyin keyingi darajaga o'tadi va B va C ga tashrif buyuradi, so'ngra

keyingi darajaga sayohat va D, E, F va G ga tashrif buyuring. Bu erda maqsad holati F sifatida belgilanadi, shuning uchun o'tish F da to'xtaydi.



O'tish yo'li:

A → B → C → D → E → F

Keling, Python dasturlashda ham xuddi shunday amal qilaylik .

Python kodi:

```

graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : [],
    'E' : [],
    'F' : [],
    'G' : []
}
visited = []
queue = []
goal = 'F'

def bfs(visited, graph, node):
    visited.append(node)
    queue.append(node)
    while queue:

```

```

s = queue.pop(0)
print (s, end = "\n")
for neighbour in graph[s]:
    if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)
    if goal in visited:
        break
bfs(visited, graph, 'A')

```

BFS ning afzalliklari

- BFS hech qachon kiruvchi tugunlarga tushmaydi.
- Agar grafikda bir nechta yechim bo'lsa, BFS eng qisqa yo'l bilan optimal echimni qaytaradi.

BFS ning kamchiliklari

- BFS joriy darajadagi barcha tugunlarni saqlaydi va keyin ularni keyingi darajaga o'tkazadi. Bu juda ko'p xotirani talab qiladi.
- BFS
uzoqda joylashgan maqsad holatiga erishish uchun ko'proq vaqt talab etadi.

2. Chuqurlik-birinchi qidiruv

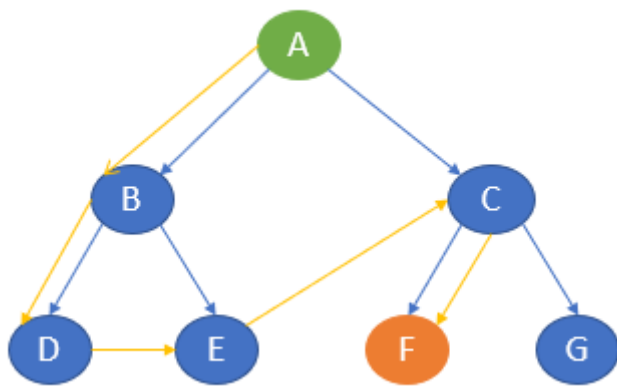
Chuqurlikdagi birinchi qidiruv oxirgi kirish, birinchi chiqish (LIFO) strategiyasidan foydalanadi va shuning uchun uni stek yordamida amalga oshirish mumkin. DFS orqaga qaytishdan foydalanadi. U dastlabki holatdan boshlanadi va keyingi yo'lga o'tishdan oldin har bir yo'lni eng katta chuqurlikka o'rganadi.

DFS kuzatib boradi

Ildiz tugun —> Chap tugun —> O'ng tugun

Endi yuqorida aytib o'tilgan daraxt misolini ko'rib chiqing.

Bu erda u A boshlang'ich holatidan boshlanadi, so'ngra B ga boradi, so'ngra D ga boradi. D ga yetgandan so'ng u B ga qaytadi. B allaqachon tashrif buyurilgan, shuning uchun u keyingi chuqurlikka, E ga boradi va keyin B ga qaytadi. U allaqachon tashrif buyurilganidek, A ga qaytadi.



O'tish yo'li:

A —> B —> D —> E —> C —> F

```
Let's try to code it.
graph = {
    'A': ['B','C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
goal = 'F'
visited = set()
def dfs(visited, graph, node):
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            if goal in visited:
```

```
        break
    else:
        dfs(visited, graph, neighbour)
dfs(visited, graph, 'A')
```

Chiqish yo'li quyidagicha.

```
A
B
D
E
C
F
```

DFS ning afzalliklari

- BFS bilan solishtirganda kamroq xotira talab qiladi.
- Vaqt murakkabligi BFS bilan solishtirganda kamroq.
- DFS ko'proq qidirishni talab qilmaydi.

DFS ning kamchiliklari

- DFS har doim ham yechim berishga kafolat bermaydi.
- DFS chuqur pastga tushar ekan, u cheksiz tsiklga tushib qolishi mumkin.

3. Chuqurlik chegaralangan qidiruv

Chuqurlik bilan cheklangan qidiruv chuqurlikdagi birinchi qidiruvga o'xshash ishlaydi. Farqi shundaki, chuqurlik bilan cheklangan qidiruv tugunlarni bosib o'tishi mumkin bo'lgan oldindan belgilangan chegaraga ega. Chuqurlik bilan cheklangan qidiruv DFS ning kamchiliklaridan birini hal qiladi, chunki u cheksiz yo'lga chiqmaydi.

Agar quyidagi shartlardan birortasi chiqsa, DFS o'z harakatini tugatadi.

Standart nosozlik

Bu berilgan muammoning yechimlari yo'qligini bildiradi.

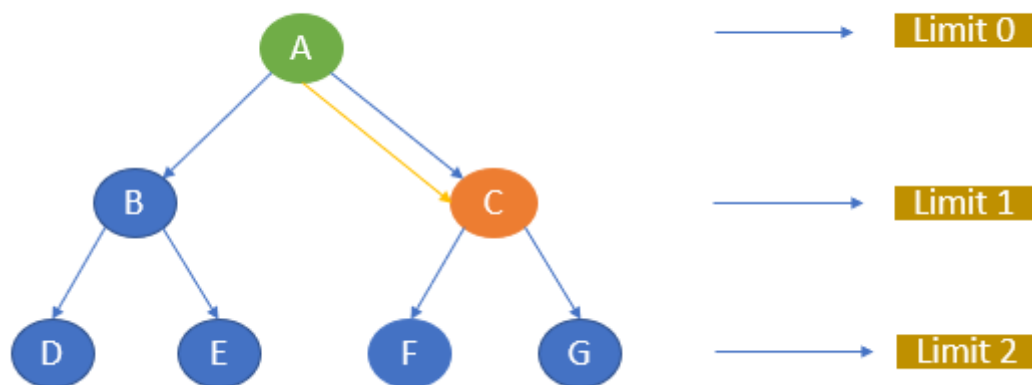
Muvaffaqiyatsizlik qiymatini kesib tashlang

Bu berilgan chegara doirasida muammoning yechimi yo'qligini bildiradi.

Endi xuddi shu misolni ko'rib chiqing.

Keling, boshlang'ich tugun sifatida A ni va maqsad holati sifatida C ni olaylik va 1 deb cheklaymiz.

O'tish A tugunidan boshlanadi va keyingi bosqichga o'tadi, 1. Maqsad holati C mavjud va u o'tishni to'xtatadi.



O'tish yo'li:

A → C

Agar maqsad tugun sifatida C ni va chegarani 0 deb bersak, algoritm hech qanday yo'lni qaytarmaydi, chunki maqsad tugun berilgan chegarada mavjud emas.

Agar maqsad tugunini F, chegarani 2 deb bersak, yo'l A, C, F bo'ladi.

Keling, DLS ni joriy qilaylik.

```

graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : [],
    'E' : [],
    'F' : [],
    'G' : []
}

def DLS(start,goal,path,level,maxD):
    print('nCurrent level-->',level)
    path.append(start)
    if start == goal:
        print("Goal test successful")
        return path
    print('Goal node testing failed')
    if level==maxD:
        return False
    print('nExpanding the current node',start)
    for child in graph[start]:
        if DLS(child,goal,path,level+1,maxD):
            return path
    path.pop()
    return False

start = 'A'
goal = input('Enter the goal node:-')
maxD = int(input("Enter the maximum depth limit:-"))
print()
path = list()
res = DLS(start,goal,path,0,maxD)
if(res):
    print("Path to goal node available")
    print("Path",path)
else:
    print("No path available for the goal node in given depth limit")

```

Maqsad tugunini C va chegara sifatida 1 ni berganimizda, yo'l quyidagicha bo'ladi.

```
Enter the goal node:-C
Enter the maximum depth limit:-1

Current level--> 0
Goal node testing failed

Expanding the current node A

Current level--> 1
Goal node testing failed

Current level--> 1
Goal test successful
Path to goal node available
Path ['A', 'C']
```

DLS ning afzalliklari

- Boshqa qidiruv usullariga qaraganda kamroq xotira talab qiladi.

DLS ning kamchiliklari

- Agar muammo bir nechta yechimga ega bo'lsa, DLS optimal echimni taklif qilmasligi mumkin.
- DLS ham to'liqsizlikka duch keladi.

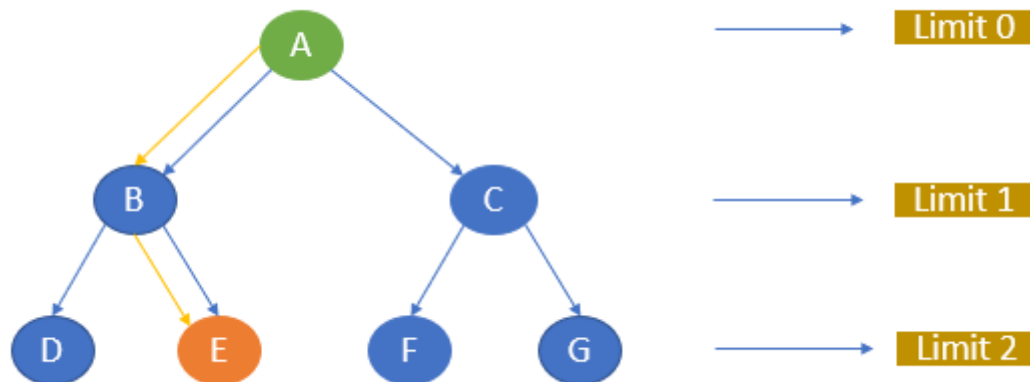
4. Takroriy chuqurlashtirish chuqurlik-birinchi qidiruv

Takroriy chuqurlashtirish chuqurlik-birinchi qidiruv chuqurlik-birinchi qidiruv va kenglik-birinchi qidiruvning birikmasidir. IDDFS belgilangan maqsad holatiga erishilgunga qadar chegarani asta-sekin qo'shish orqali eng yaxshi chuqurlik chegarasini topadi.

Keling, buni xuddi shu misol daraxti yordamida tushuntirishga harakat qilaylik.

A ni boshlang'ich tugun sifatida va E ni maqsad tugun sifatida ko'rib chiqing. Maksimal chuqurlik 2 bo'lsin.

Algoritm A dan boshlanadi, keyingi bosqichga o'tadi va E ni qidiradi. Agar E ni topmasa, keyingi bosqichga o'tadi va uni yana topadi.



O'tish yo'li

A → B → E

Keling, buni amalga oshirishga harakat qilaylik.

```
graph = {
    'A': ['B','C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}
path = list()
def DFS(currentNode,destination,graph,maxDepth,curList):
    curList.append(currentNode)
    if currentNode==destination:
        return True
    if maxDepth<=0:
        path.append(curList)
        return False
    for node in graph[currentNode]:
        if DFS(node,destination,graph,maxDepth-1,curList):
            return True
    else:
        curList.pop()
    return False
def iterativeDDFS(currentNode,destination,graph,maxDepth):
    for i in range(maxDepth):
        curList = list()
```



```

        if DFS(currentNode,destination,graph,i,curList):
            return True
        return False
if not iterativeDDFS('A','E',graph,3):
    print("Path is not available")
else:
    print("Path exists")
    print(path.pop())

```

Yaratilgan

yo'l

quyidagicha.

```

Path exists
['A', 'B', 'E']

```

IDDFS ning afzalliklari

- IDDFS ham BFS, ham DFS afzalliklariga ega.
- U tezkor qidiruvni taklif qiladi va xotiradan samarali foydalanadi.

IDDFS ning kamchiliklari

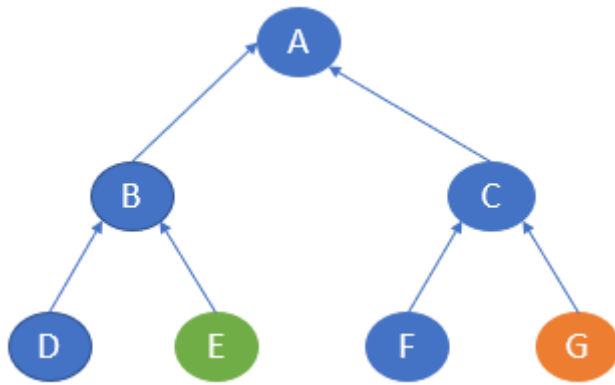
- U avvalgi bosqichdagi barcha ishlarni qayta-qayta bajaradi.

5. Ikki tomonlama qidiruv

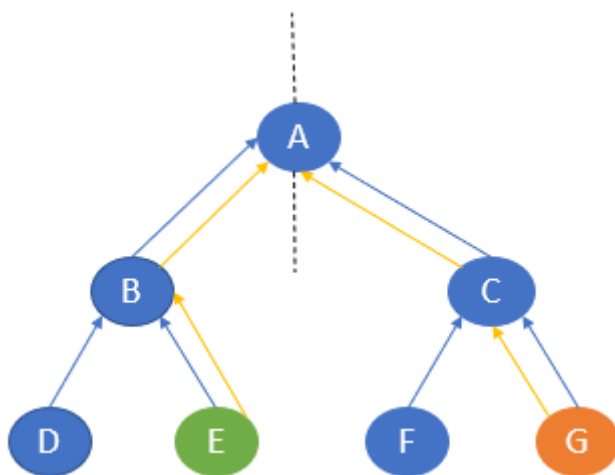
Ikki tomonlama qidiruv algoritmi boshqa barcha qidiruv strategiyalaridan butunlay farq qiladi. U oldinga va orqaga qidirish deb ataladigan bir vaqtning o'zida ikkita qidiruvni amalga oshiradi va maqsad holatiga etadi. Bu erda grafik ikkita kichik kichik grafikaga bo'lingan. Bir grafikda qidiruv dastlabki boshlang'ich holatidan boshlanadi, boshqa grafikda esa maqsad holatidan boshlanadi. Ushbu ikkita tugun kesishganda, qidiruv to'xtatiladi.

Ikki yo'nalishli qidiruv ikkala yo'nalishda ham boshlang'ich, ham maqsad boshlanishini va bir xil tarmoqlanish omilini aniq belgilashni talab qiladi.

Quyidagi grafikni ko'rib chiqing.



Bu erda boshlang'ich holati E, maqsad holati esa G. Bir kichik grafikda qidiruv E dan boshlanadi, ikkinchisida esa G dan boshlanadi. E B ga va keyin A ga, G C ga va keyin A ga boradi. Bu erda ikkala o'tish A da uchrashadi va shuning uchun o'tish tugaydi.



O'tish yo'li

E → B → A → C → G

Keling, xuddi shu narsani Python-da amalga oshiramiz.

```

from collections import deque
class Node:
    def __init__(self, val, neighbors=[]):
        self.val = val
        self.neighbors = neighbors
        self.visited_right = False
        self.visited_left = False
        self.parent_right = None
  
```

```

        self.parent_left = None
def bidirectional_search(s, t):
    def extract_path(node):
        node_copy = node
        path = []
        while node:
            path.append(node.val)
            node = node.parent_right
        path.reverse()
        del path[-1]
        while node_copy:
            path.append(node_copy.val)
            node_copy = node_copy.parent_left
        return path
    q = deque([s])
    q.append(s)
    q.append(t)
    s.visited_right = True
    t.visited_left = True
    while len(q) > 0:
        n = q.pop()
        if n.visited_left and n.visited_right:
            return extract_path(n)
        for node in n.neighbors:
            if n.visited_left == True and not node.visited_left:
                node.parent_left = n
                node.visited_left = True
                q.append(node)
            if n.visited_right == True and not node.visited_right:
                node.parent_right = n
                node.visited_right = True
                q.append(node)
    return False
n0 = Node('A')
n1 = Node('B')
n2 = Node('C')
n3 = Node('D')
n4 = Node('E')
n5 = Node('F')
n6 = Node('G')
n0.neighbors = []
n1.neighbors = [n0]
n2.neighbors = [n0]
n3.neighbors = [n1]
n4.neighbors = [n1]
n5.neighbors = [n2]
n6.neighbors = [n2]
print(bidirectional_search(n4, n6))

```

Yo'l quyidagicha yaratiladi.

```
['E', 'B', 'A', 'C', 'G']
```

Ikki tomonlama qidiruvning afzalliklari

- Ushbu algoritm grafikni tezda qidiradi.
- Uning harakatini bajarish uchun kamroq xotira talab qilinadi.

Ikki tomonlama qidiruvning kamchiliklari

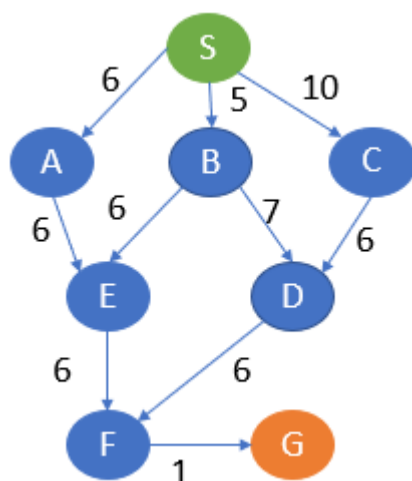
- Maqsad holati oldindan belgilanishi kerak.
- Grafikni

amalga oshirish juda qiyin.

6. Yagona xarajatlarni qidirish

Yagona xarajatlarni qidirish vaznli grafik yoki xarajatlar bilan grafik uchun eng yaxshi qidiruv algoritmi hisoblanadi. U eng past umumiy xarajatga maksimal ustunlik berib, grafikni qidiradi. Yagona xarajatlarni qidirish ustuvor navbat yordamida amalga oshirilishi mumkin.

Quyidagi grafikni ko'rib chiqing, bu erda har bir tugun oldindan belgilangan narxga ega.

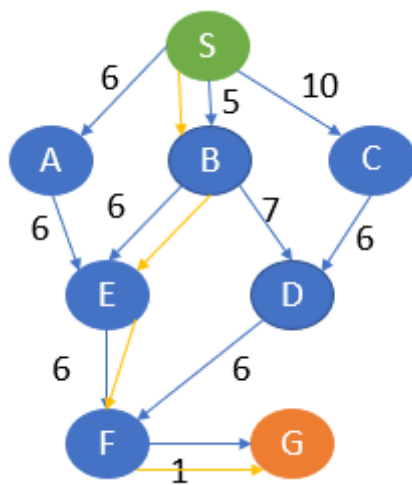


Bu erda S boshlang'ich tugun, G esa maqsad tugunidir.

S, G dan quyidagi yo'llar bilan erishish mumkin.

- S, A, E, F, G -> 19
- S, B, E, F, G -> 18
- S, B, D, F, G -> 19
- S, C, D, F, G -> 23

Bu erda eng kam xarajatli yo'l S, B, E, F, G.



Keling, Pythonda UCS ni joriy qilaylik.

```

graph=[['S','A',6],
        ['S','B',5],
        ['S','C',10],
        ['A','E',6],
        ['B','E',6],
        ['B','D',7],
        ['C','D',6],
        ['E','F',6],
        ['D','F',6],
        ['F','G',1]]
temp = []
temp1 = []
for i in graph:
    temp.append(i[0])
    temp1.append(i[1])
nodes = set(temp).union(set(temp1))
def UCS(graph, costs, open, closed, cur_node):
    if cur_node in open:
        open.remove(cur_node)
        closed.add(cur_node)
    for i in graph:

```

```

    if(i[0] == cur_node and costs[i[0]]+i[2] < costs[i[1]]):
        open.add(i[1])
        costs[i[1]] = costs[i[0]]+i[2]
        path[i[1]] = path[i[0]] + ' -> ' + i[1]
    costs[cur_node] = 999999
    small = min(costs, key=costs.get)
    if small not in closed:
        UCS(graph, costs, open, closed, small)
costs = dict()
temp_cost = dict()
path = dict()
for i in nodes:
    costs[i] = 999999
    path[i] = ''
open = set()
closed = set()
start_node = input("Enter the Start State: ")
open.add(start_node)
path[start_node] = start_node
costs[start_node] = 0
UCS(graph, costs, open, closed, start_node)
goal_node = input("Enter the Goal State: ")
print("Path with least cost is: ", path[goal_node])

```

Optimal chiqish yo'li yaratiladi.

```

Enter the Start State: S
Enter the Goal State: G
Path with least cost is:  S -> B -> E -> F -> G

```

UCS ning afzalliklari

- Ushbu algoritm optimal hisoblanadi, chunki yo'llarni tanlash eng past narxga asoslangan.

UCS ning kamchiliklari

- Algoritm
eng past yo'lga erishish uchun qancha qadam kerakligini hisobga olmaydi. Bu cheksiz tsiklga ham olib kelishi mumkin.

Turli xil ma'lumotsiz qidiruv algoritmlarini solishtirish

Keling, vaqtning murakkabligiga asoslangan oltita ma'lumotsiz qidiruv strategiyasini solishtirishga ijozat bering.

Algoritm	Vaqt	Kosmos	Bajarildi	Opti
Kenglik birinchi	$O(b^d)$	$O(b^d)$	Ha	Ha
Birinchi chuqurlik	$O(b^m)$	$O(bm)$	Yo'q	Yo'q
Chuqurlik cheklangan	$O(b^l)$	$O(bl)$	Yo'q	Yo'q
Iterativ chuqurlashtirish	$O(b^d)$	$O(bd)$	Ha	Ha
Ikki tomonlama	$O(b^{(d/2)})$	$O(b^{(d/2)})$	Ha	Ha
Yagona xarajat	$O(bl + \lceil \log(C^*/\epsilon) \rceil \cdot b)$	$O(bl + \lceil \log(C^*/\epsilon) \rceil \cdot b)$	Ha	Ha

Bularning barchasi ma'lumotsiz qidiruv algoritmlari haqida.

Keling, ma'lumotli qidiruv algoritmlarini ko'rib chiqaylik.

Axborotlashtirilgan qidiruv algoritmlari

Axborotlashtirilgan qidiruv algoritmi evristik qidiruv yoki yo'naltirilgan qidiruv deb ham ataladi. Ma'lumotga ega bo'lmagan qidiruv algoritmlaridan farqli o'laroq, ma'lumotli qidiruv algoritmlari maqsadga erishish uchun masofa, maqsadga erishish uchun qadamlar va yo'llarning narxi kabi tafsilotlarni talab qiladi, bu esa ushbu algoritmni samaraliroq qiladi.

Bu erda maqsad holatiga evristik funktsiyadan foydalanish orqali erishish mumkin.

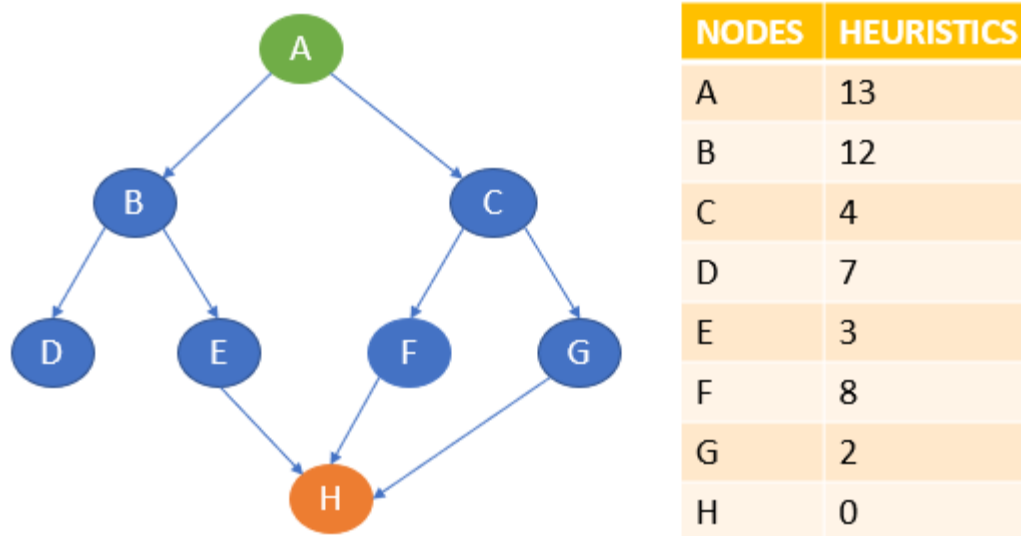
Evristik funktsiya maqsad holatiga eng kam xarajat bilan erishadi. Bu funktsiya davlatning maqsadga qanchalik yaqinligini taxmin qiladi.

Keling, ba'zi ma'lumotli qidiruv strategiyalarini muhokama qilaylik.

1. Greedy eng yaxshi birinchi qidiruv algoritmi

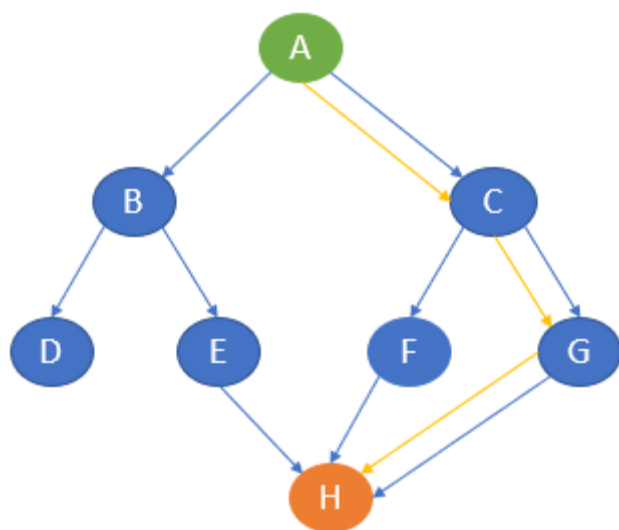
Ochko'zlik bilan birinchi bo'lib qidiruv ham chuqurlikdan-birinchi qidiruvdan, ham kenglikdan-birinchi qidiruvdan foydalanadi. Ayni paytda eng yaxshi ko'rinadigan yo'lni tanlab, tugunni kesib o'tadi. Eng yaqin yo'l evristik funktsiya yordamida tanlanadi.

Quyidagi grafikni evristik qiymatlari bilan ko'rib chiqing.



Bu erda A boshlang'ich tugun, H esa maqsad tugunidir.

Ochko'zlik eng yaxshi birinchi qidiruvi A bilan boshlanadi va keyingi qo'shnilarni, B va C ni tekshiradi. Bu erda B ning evristikasi 12 va C 4 ga teng. Hozirda eng yaxshi yo'l C, shuning uchun u C ga boradi. C dan F va G qo'shnilarini o'rganadi. F ning evristikasi 8 va G 2, shuning uchun u G dan kimga ketadi, u G ga boradi. ham bizning maqsadli davlatimizdir.



O'tish yo'li

A → C → G → H

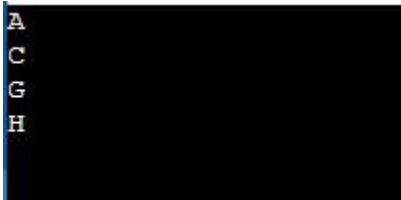
Keling, buni Python bilan sinab ko'raylik.

```

graph = {
    'A': [('B', 12), ('C', 4)],
    'B': [('D', 7), ('E', 3)],
    'C': [('F', 8), ('G', 2)],
    'D': [],
    'E': [('H', 0)],
    'F': [('H', 0)],
    'G': [('H', 0)]
}

def bfs(start, target, graph, queue=[], visited=[]):
    if start not in visited:
        print(start)
        visited.append(start)
        queue = queue + [x for x in graph[start] if x[0] not in visited]
        queue.sort(key=lambda x: x[1])
        if queue[0][0] == target:
            print(queue[0][0])
        else:
            processing = queue[0]
            queue.remove(processing)
            bfs(processing[0], target, graph, queue, visited)
bfs('A', 'H', graph)
  
```

Eng kam xarajat bilan chiqish yo'li ishlab chiqariladi.



Greedy best-first qidiruvining vaqt murakkabligi eng yomon holatlarda $O(b^m)$ dir.

Greedy eng yaxshi birinchi qidiruvning afzalliklari

- Ochko'zlik bilan birinchi bo'lib qidiruv kenglik-birinchi qidiruv va chuqurlik-birinchi qidiruv bilan solishtirganda samaraliroq.

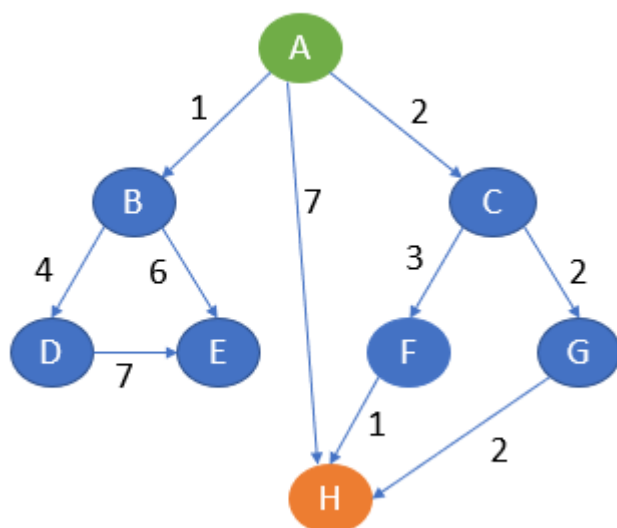
Greedy eng yaxshi birinchi qidiruvning kamchiliklari

- Eng yomon stsenariyda ochko'z eng yaxshi birinchi qidiruv algoritmi boshqarilmaydigan DFS kabi harakat qilishi mumkin.
- Ochko'zning ba'zi imkoniyatlari bor - birinchi navbatda cheksiz halqaga tushib qolish.
- Algoritm optimal emas.

Keyinchalik, A* qidiruv algoritmi deb ataladigan boshqa ma'lumotli qidiruv algoritmini muhokama qilaylik.

2. A* qidiruv algoritmi

A* qidiruv algoritmi yagona xarajat qidirish va ochko'z eng yaxshi birinchi qidiruv algoritmlarining kombinatsiyasidir. Bu ikkalasining afzalliklarini yaxshi xotiradan foydalanish bilan birlashtiradi. Eng qisqa yo'lni topish uchun evristik funksiyadan foydalanadi. A* qidiruv algoritmi eng yaxshi yo'lni topish uchun tugun narxi va evristik ma'lumotlar yig'indisidan foydalanadi.



NODES	HEURISTICS
A	5
B	3
C	4
D	2
E	6
F	3
G	1
H	0

A boshlang'ich tugun, H esa maqsad tugun bo'lsin.

Algoritm A bilan boshlanadi. A dan B, C yoki H ga o'tishi mumkin.

E'tibor bering, A* qidiruvi yo'lni aniqlash uchun yo'l narxi va evristik qiymat yig'indisidan foydalanadi.

Bu erda, A dan B gacha, xarajat va evristika yig'indisi $1 + 3 = 4$ ga teng.

A dan C gacha, u $2 + 4 = 6$ ga teng.

A dan H gacha, u $7 + 0 = 7$ ga teng.

Bu erda eng kam xarajat 4 ga teng bo'lib, A dan B gacha bo'lgan yo'l tanlanadi. Boshqa yo'llar to'xtatiladi.

Endi B dan D yoki E ga o'tishi mumkin.

Narxi $1 + 4 + 2 = 7$ A dan B dan D gacha.

A dan B dan E gacha, $1 + 6 + 6 = 13$.

Eng kam xarajat 7. A dan B dan D gacha bo'lgan yo'l tanlanadi va boshqa kutish yo'llari bilan solishtiriladi.

Bu erda A dan C gacha bo'lgan yo'llar arzonroq. Bu 6.

Shunday qilib, A dan C gacha tanlanadi va boshqa yo'llar to'xtatiladi.

C dan endi u F yoki G ga o'tishi mumkin.

A dan C dan F gacha, xarajat $2 + 3 + 3 = 8$ ni tashkil qiladi.

Narxi $2 + 2 + 1 = 5$ A dan C dan G gacha.

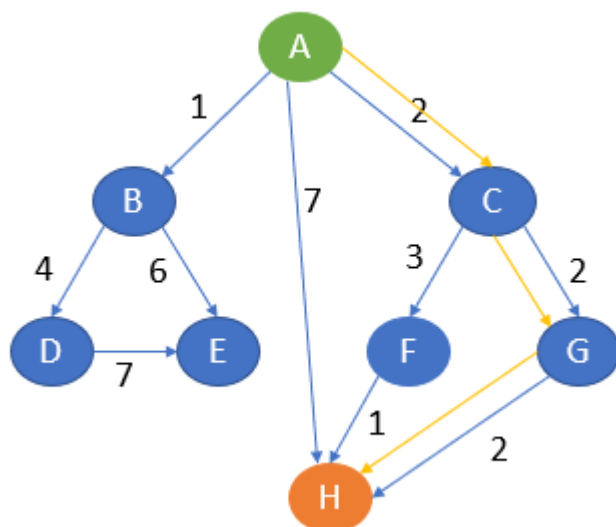
Eng kam xarajat 5 ni tashkil etadi, bu ham kutishdagi boshqa yo'llarga qaraganda kamroq.

Demak, A dan G gacha bo'lgan yo'llar tanlanadi.

G dan u H ga o'tishi mumkin, uning narxi $2 + 2 + 2 + 0 = 6$.

Bu erda, 6 boshqa yo'llarning narxidan kamroq, bu kutishda.

Bundan tashqari, H bizning maqsad holatimizdir. Algoritm shu yerda tugaydi.



Python

Keling, buni Pythonda sinab ko'raylik.

```
graph=[['A','B',1,3],
        ['A','C',2,4],
        ['A','H',7,0],
        ['B','D',4,2],
        ['B','E',6,6],
        ['C','F',3,3],
        ['C','G',2,1],
        ['D','E',7,6],
        ['D','H',5,0],
        ['F','H',1,0],
        ['G','H',2,0]]
temp = []
temp1 = []
for i in graph:
    temp.append(i[0])
    temp1.append(i[1])
nodes = set(temp).union(set(temp1))
def A_star(graph, costs, open, closed, cur_node):
    if cur_node in open:
        open.remove(cur_node)
    closed.add(cur_node)
    for i in graph:
        if(i[0] == cur_node and costs[i[0]]+i[2]+i[3] < costs[i[1]]):
            open.add(i[1])
            costs[i[1]] = costs[i[0]]+i[2]+i[3]
            path[i[1]] = path[i[0]] + '->' + i[1]
    costs[cur_node] = 999999
    small = min(costs, key=costs.get)
    if small not in closed:
        A_star(graph, costs, open, closed, small)
costs = dict()
temp_cost = dict()
path = dict()
for i in nodes:
    costs[i] = 999999
    path[i] = ''
open = set()
closed = set()
start_node = input("Enter the Start Node: ")
open.add(start_node)
path[start_node] = start_node
costs[start_node] = 0
A_star(graph, costs, open, closed, start_node)
goal_node = input("Enter the Goal Node: ")
print("Path with least cost is: ",path[goal_node])
```

Chiqish quyidagicha berilgan

```
Enter the Start Node: A
Enter the Goal Node: H
Path with least cost is: A -> C -> G -> H
```

A* qidiruvining vaqt murakkabligi $O(b^d)$, bu yerda b tarmoqlanish omili.

A* qidiruv algoritmining afzalliklari

- Bu algoritm boshqa algoritmlar bilan solishtirganda eng yaxshisidir.
- Bu algoritmdan juda murakkab masalalarni yechishda foydalanish mumkin va u optimal hisoblanadi.

A* qidiruv algoritmining kamchiliklari

- A* qidiruvi evristik va narxga asoslangan. Bu eng qisqa yo'lni yaratmasligi mumkin.
- Xotiradan foydalanish ko'proq, chunki u barcha tugunlarni xotirada saqlaydi.

Keling, ma'lumotsiz va ma'lumotli qidiruv strategiyalarini taqqoslaylik.

Axborotsiz va ma'lumotli qidiruv algoritmlarini solishtirish

Axborotsiz qidiruv, shuningdek, ko'r-ko'rona qidiruv sifatida ham tanilgan, ma'lumotli qidiruv esa evristik qidiruv deb ham ataladi. Yagona qidiruv juda ko'p ma'lumotni talab qilmaydi, ma'lumotli qidiruv esa domenga xos tafsilotlarni talab qiladi. Axborotsiz qidiruv bilan solishtirganda, ma'lumotga asoslangan qidiruv strategiyalari samaraliroq va ma'lumotsiz qidiruv strategiyalarining vaqt murakkabligi kattaroqdir. Axborotli qidiruv muammoni ko'r-ko'rona qidirishdan ko'ra yaxshiroq hal qiladi.

Xulosa

Qidiruv algoritmlari o'yinlarda, saqlangan ma'lumotlar bazalarida, virtual qidiruv maydonlarida, kvant kompyuterlarida va hokazolarda qo'llaniladi. Ushbu maqolada biz ba'zi muhim qidiruv strategiyalarini va AI muammolarini hal qilish uchun ulardan qanday foydalanishni muhokama qildik; bu oxiri emas. Bir nechta algoritmlar har qanday muammoni hal qilishi mumkin. Hozirgi vaqtda sun'iy intellekt tez sur'atlar bilan o'sib bormoqda va ko'plab real hayot muammolariga taalluqlidir. O'rganishda davom eting! Mashq qilishda davom eting!